List 1: Python basics for Java programmers

compiled by Witold Dyrka, Ewelina Tomana

8th October 2026

General information

All code should be written according to The Style Guide for Python Code.

All functions should be placed in a Python module named `list1`. Each function should be described with a docstring (see Docstring Conventions). The docstring should briefly describe what the function does, what are input arguments (name, type, meaning), and what the function returns (type, meaning). It should also provide pre- and postconditions. We recommend sticking to the Google or NumPy documentation comment styles.[1]

If pre- or postconditions or *invariants* are not met, the code should raise an *exception*.

There should be another module `test_list1` that *imports* module `list1`, and defines (and runs) tests for its functions. Each non-trivial function should be tested using typical and boundary values of its arguments. Raising exceptions in case of invalid arguments should also be tested.

All programs (here: Exercise 6) should communicate with the user in intelligible manner, including handling the exceptions.

Carefully avoid plagiarism. Each external source of code should be precisely referenced. The use of prompt-based artificial intelligence tools to support completing problem sets is permitted provided that *full* documentation of the code development process is submitted, including specifying all tools and models used, their versions, and all prompts. The use of plugin-based coding assistants is prohibited.

Exercise 1

Write function `divisible(numbers, divisor)` that takes as arguments a list of integers, `numbers`, and an integer, `divisor`, and returns a list of only these `numbers` that are divisible by `divisor`.

Exercise 2

From *Programming & classification* by Marek Klonowski, adapted.

Write a function `common(x, y)` that, for two multisets x and y, returns their intersection.

[1] A concise comparison of both styles can be found, among others, in the documentation for the Sphinx documentation generator at https://www.sphinx-doc.org/en/master/usage/extensions/napoleon.html

Exercise 3

From *Programming & classification* by Marek Klonowski, adapted.

Write function `subsets(x)` that, for a given set x, returns a list of all its subsets. For example, for `x = {a, b, c, d}`, the function should return `{a, b, c, d}`, `{a, b, c}`, `{a, b, d}`, `{a, c, d}`, `{b, c, d}`, `{a, b}`, `{a, c}`, `{a, d}`, `{b, c}`, `{b, d}`, `{c, d}`, `{a}`, `{b}`, `{c}`, `{d}`, `{}`, in any order.

Exercise 4

From *Introduction to programming* by Klaudia Kozłowska, Mirosław Łątka, adapted.

Write function `random_walk(n)` that implements a two-dimensional random walk. In this scenario, a random walker begins its journey at the centre of the coordinate system $(0, 0)$. During each step, it has the option to randomly move one unit to the left, right, up, or down. The coordinates are continuously recorded as a list of 2-tuples:

```
[(x_1, y_1), (x_2, y_2), (x_3, y_3), ... ]
```

until the walker is more than n units away from the centre. Visualise the walker's trajectory using the *matplotlib* package. You can use the following code:

```
import matplotlib.pyplot as plt

n = 100
coordinates = random_walk(n)
x, y = zip(*coordinates)

plt.plot(x, y, marker='o', linestyle='-', markersize=2)
plt.xlabel('X Coordinate')
plt.ylabel('Y Coordinate')
plt.title('Walker\'s Trajectory')
plt.grid(True)
plt.show()
```

Remember to install *matplotlib* in your Python environment.

Exercise 5

From *Introduction to programming* by Klaudia Kozłowska, Mirosław Łątka, adapted.

Genetic information is encoded in DNA using just four bases: guanine, cytosine, adenine, and thymine, commonly referred to as G, C, A, and T, respectively. Two DNA strands twist around each other to form a structure called a double helix. The bases on each strand pair up: A bonds with T, and C bonds with G.

Write function `dna_complement(orig_strand)` that for given `orig_strand` returns its complementary strand using the Watson-Crick pairing rules. For example, if the input argument is `ATCG`, the output of the function should be `CGAT`.

Exercise 6

From *Introduction to programming* by Mirosław Łątka, adapted.

A gene is a substring of a genome that represents a functional unit of critical importance in understanding life processes. A gene consists of a sequence of codons, each of which is a sequence of three bases representing one amino acid. The start codon `ATG` marks the beginning of a gene, and any of the stop codons `TAG`, `TAA`, or `TGA` marks the end of a gene.

Identifying potential genes within a genome is an essential task in genome analysis, which involves various string processing techniques. To address this task, in a separate script `find_genes.py`, write a program that asks the user to provide a DNA sequence (string) and lists all potential genes in this sequence *and* in its complementary sequence. To be considered a gene, a subsequence has to meet the following conditions:

- it starts with the start codon `ATG`,
- it ends with a stop codon,
- its length is a multiple of 3,
- it does not contain any intervening stop codons.